

Contents

RFLR 2026 — Lectures 4, 5, 6: Finding the Optimal Policy	3
Part 0 — Where we are	3
PART 1 — Lecture 4: the operator $B\pi$	5
1.1 The problem it solves	5
1.2 Definition	5
1.3 $V\pi$ is the fixed point	6
1.4 Property 1: Monotonicity	6
1.5 Property 2: Contraction	7
1.6 The algorithm: iterative policy evaluation	8
1.7 Convergence, with the rate	8
PART 2 — The five one-step backups (memorise this)	10
PART 3 — Lecture 5: Q-values and Policy Iteration	12
3.1 Q-values	12
3.2 The Policy Iteration algorithm	13
3.3 The Policy Improvement Theorem	14
3.4 Why Policy Iteration terminates — and terminates at the optimum	15
3.5 V^* exists (is finite)	16
3.6 V^* is unique	17
PART 4 — Lecture 6: the Bellman Optimality Equation and Value Iteration	20
4.1 Greedy policy	20
4.2 The Bellman Optimality Equation	20
4.3 The operator B	21
4.4 Property 1: B is monotone	22
4.5 Property 2: B is a contraction	22
4.6 Value Iteration	24

4.7 Stopping criterion in practice	25
4.8 How far are we from V^* ? (Maam’s “to do” — prove this)	25
PART 5 — Policy Iteration vs Value Iteration	28
PART 6 — Fully worked examples	30
6.1 Robot MDP — Policy Iteration	30
6.2 Robot MDP — Value Iteration	33
6.3 HW1 MDP — all 8 policies, and a surprise	35
6.4 HW1 MDP — Policy Iteration	36
6.5 HW1 MDP — Value Iteration, and the tightness of the error bound	39
PART 7 — Practice problems	41
PART 8 — Cheat sheet	43
Common traps	46
Reading	47

RFLR 2026 — Lectures 4, 5, 6: Finding the Optimal Policy

Covers: Bellman backup operators, Q-values, Policy Iteration, the Bellman Optimality Equation, Value Iteration

Part 0 — Where we are

Lectures 1–3 answered one question: **given a policy π , what is it worth?** That's V_π , and you now have three ways to get it (definition, recursion, matrix inverse).

Lectures 4–6 answer a much bigger question: **which policy is best?**

The bridge between them is one idea: **operators**. Instead of thinking about V_π as a number you solve for, you think about a *machine* that takes a value vector and improves it. Everything in Lectures 4–6 is built from three such machines:

Machine	Introduced	What it does
B_π	Lecture 4	one step of “follow π ” — iterating it gives V_π
Q_π / QV	Lecture 5–6	one step of “try action a , then follow the policy / trust V ”
B	Lecture 6	one step of “take the best action” — iterating it gives V^*

Two algorithms fall out:

- **Policy Iteration** (Lecture 5) — walk through policy space: $\pi_0 \rightarrow \pi_1 \rightarrow \pi_2 \rightarrow \dots \rightarrow \pi^*$
- **Value Iteration** (Lecture 6) — walk through value space: $V_0 \rightarrow V_1 \rightarrow V_2 \rightarrow \dots \rightarrow V^*$

(Linear Programming is the third algorithm she listed; it hasn't been covered yet. A one-paragraph preview is in Part 6 so you know where it fits.)

Read in this order. Part 1 rebuilds Lecture 4 from scratch — don't skip it even though you've seen the slides, because Lecture 5's main proof is *entirely* Lecture 4's two properties applied twice. Part 2 is the single table that organises everything; if you memorise one thing, memorise that.

PART 1 — Lecture 4: the operator B_π

1.1 The problem it solves

You know $V_\pi = (I - \gamma P_\pi)^{-1} R_\pi$. So why do we need another method?

Because inverting an $n \times n$ matrix costs about n^3 operations and needs n^2 memory. For $n = 3$, fine. For a real MDP with a million states, the matrix has 10^{12} entries — you cannot even store it, let alone invert it. We need a method that never forms the inverse.

The trick: **turn the equation into a repeated update.**

1.2 Definition

$B_\pi : \mathbb{R}^n \rightarrow \mathbb{R}^n$ is the **Bellman backup operator for policy π** :

$$B_\pi[V] = R_\pi + \gamma P_\pi V \quad \text{for any } V \in \mathbb{R}^n$$

Componentwise:

$$B_\pi[V](s) = R(s, \pi(s)) + \gamma \sum_{s' \in S} P_{ss'}(\pi(s)) \cdot V(s')$$

Read it as an instruction: “take one real step under π , collect the real reward, and then — instead of continuing — just look up your current guess V for wherever you landed.”

The distinction that trips everyone up

Symbol	What it is
V_π	the value function of π . One specific, true vector.

V any vector in $\mathbb{R}^n$. A guess. Could be zeros, could be nonsense.

$B_\pi[V]$ the result of improving that guess by one step.

B_π accepts garbage and returns slightly-less-garbage. That's the whole algorithm.

1.3 V_π is the fixed point

The Bellman equation from Lecture 3 says $V_\pi = R_\pi + \gamma P_\pi V_\pi$. Look at the right-hand side — it is B_π applied to V_π . So:

$$B_\pi[V_\pi] = V_\pi$$

V_π is the one vector that B_π leaves alone. Everything else, it moves — and (as we'll prove) it moves it toward V_π .

1.4 Property 1: Monotonicity

$$V_1 \leq V_2 \implies B_\pi[V_1] \leq B_\pi[V_2] \quad (\text{componentwise})$$

Proof.

$$B_\pi[V_2] - B_\pi[V_1] = (R_\pi + \gamma P_\pi V_2) - (R_\pi + \gamma P_\pi V_1) = \gamma P_\pi (V_2 - V_1)$$

R_π cancels. Now $V_2 - V_1 \geq 0$ by assumption, $\gamma \geq 0$, and every entry of P_π is a probability so $P_\pi \geq 0$. A non-negative matrix times a non-negative vector is non-negative. Hence $B_\pi[V_2] - B_\pi[V_1] \geq 0$. ■

In words: if you're more optimistic about the future everywhere, your backed-up estimate can only go up. Nothing about backing up can turn better news into worse news.

This property looks trivial. It is the engine of the entire Policy Improvement Theorem in Lecture 5 — you will use it to chain an infinite sequence of inequalities.

1.5 Property 2: Contraction

$$\|B\pi[V_1] - B\pi[V_2]\|_\infty \leq \gamma \|V_1 - V_2\|_\infty$$

where $\|x\|_\infty = \max_i |x_i|$ is the largest absolute entry.

Lemma first — for any stochastic matrix P (non-negative, rows sum to 1) and any vector x :

$$|(Px)_i| = \left| \sum_j P_{ij} x_j \right| \leq \sum_j P_{ij} |x_j| \leq \sum_j P_{ij} \|x\|_\infty = \|x\|_\infty \cdot \sum_j P_{ij} = \|x\|_\infty$$

(triangle inequality $\rightarrow$ bound each $|x_j|$ by the maximum $\rightarrow$ rows sum to 1)

So $\|Px\|_\infty \leq \|x\|_\infty$. **A stochastic matrix never enlarges the max-norm.** This is worth remembering on its own; it shows up in four different proofs across these three lectures.

Proof of contraction.

$$\begin{aligned} B\pi[V_1] - B\pi[V_2] &= \gamma P\pi(V_1 - V_2) && (\text{R}\pi \\ &\text{cancels again}) \\ \|B\pi[V_1] - B\pi[V_2]\|_\infty &= \gamma \|P\pi(V_1 - V_2)\|_\infty \leq \gamma \|V_1 - V_2\|_\infty \end{aligned}$$

■

In words: applying $B\pi$ pulls any two vectors at least a factor γ closer together. γ is the *shrink factor*.

Why contraction is the whole ballgame

Contraction gives you three things at once (this is the **Banach fixed-point theorem**):

1. A fixed point **exists**.
2. It is **unique** — so $V\pi$ is well defined; there's no second candidate.
3. Iterating from **any** starting vector converges to it.

You get all of that for free from $\gamma < 1$. That's the real reason discounting is not just a modelling convenience.

1.6 The algorithm: iterative policy evaluation

```

 $V_0 \in \mathbb{R}^n$                                 (anything — zeros are
conventional)
 $k \leftarrow 0$ 
Repeat
     $V_{\{k+1\}} = B\pi[V_k]$                     i.e.  $V_{\{k+1\}} = R\pi + \gamma$ 
     $B\pi V_k$ 
     $k \leftarrow k + 1$ 
Until  $V_k = V_{\{k-1\}}$ 
Return  $V_k$ 

```

$$V_0 \xrightarrow{B\pi} V_1 \xrightarrow{B\pi} V_2 \xrightarrow{B\pi} \dots \rightarrow V\pi$$

1.7 Convergence, with the rate

Using $B\pi[V\pi] = V\pi$ and contraction repeatedly:

$$\begin{aligned}
\|V_\pi - V_1\|_\infty &= \|B\pi[V_\pi] - B\pi[V_0]\|_\infty \leq \gamma \|V_\pi - V_0\|_\infty \\
\|V_\pi - V_2\|_\infty &= \|B\pi[V_\pi] - B\pi[V_1]\|_\infty \leq \gamma \|V_\pi - V_1\|_\infty \leq \gamma^2 \|V_\pi - V_0\|_\infty \\
&\vdots \\
\|V_\pi - V_k\|_\infty &\leq \gamma^k \|V_\pi - V_0\|_\infty \rightarrow 0
\end{aligned}$$

So $V_\pi = \lim_{k \rightarrow \infty} B\pi^k[V]$ for **any** $V \in \mathbb{R}^n$.

Rate: the error shrinks by a factor γ per sweep. With $\gamma = 0.9$, $0.9^{22} \approx 0.1$, so you gain roughly one decimal digit every 22 sweeps. Big $\gamma \Rightarrow$ slow. This is exactly why Value Iteration in Lecture 6 will need dozens of sweeps while Policy Iteration needs three.

PART 2 — The five one-step backups (memorise this)

Lecture 6 slide 4 lists four of these; adding **B** makes five. **This table is the organising idea of the entire unit.** Every formula in Lectures 5 and 6 is one of these five.

Every one has the same shape: $\text{immediate reward} + \gamma \times (\text{average of something over next states})$. They differ on exactly two questions.

Question 1 — *where does the first action come from?* forced by π · freely chosen as a · maximised over all a

Question 2 — *what value do you back up onto?* the true $V\pi$ · an arbitrary guess V

That's a 3×2 grid, and five of the six cells have names:

	back up onto true $V\pi$	back up onto any V
first action forced by π	$V\pi(s)$	$B\pi[V](s)$
first action is a free a	$Q\pi(s, a)$	$QV(s, a)$
first action maximised	(= V^* when π optimal)	$B[V](s)$

Written out:

```

 $V\pi(s) = R(s, \pi(s)) + \gamma \sum_{s'} P_{ss'}(\pi(s)) \cdot V\pi(s')$ 
true value, self-consistent
 $Q\pi(s, a) = R(s, a) + \gamma \sum_{s'} P_{ss'}(a) \cdot V\pi(s')$ 
take  $a$  once, then follow  $\pi$ 
 $B\pi[V](s) = R(s, \pi(s)) + \gamma \sum_{s'} P_{ss'}(\pi(s)) \cdot V(s')$ 
one sweep of "follow  $\pi$ "
 $QV(s, a) = R(s, a) + \gamma \sum_{s'} P_{ss'}(a) \cdot V(s')$ 
take  $a$  once, then trust  $V$ 
 $B[V](s) = \max_a \{ R(s, a) + \gamma \sum_{s'} P_{ss'}(a) \cdot V(s') \}$ 
one sweep of "act greedily"

```

The relations between them — these are exam gold

$Q_{\pi}(s, \pi(s))$	$= V_{\pi}(s)$	following π IS choosing $a = \pi(s)$
$B_{\pi}[V](s)$	$= QV(s, \pi(s))$	B_{π} is QV with the action forced
$B[V](s)$	$= \max_a QV(s, a)$	B is QV with the action maximised
$B[V]$	$\geq B_{\pi}[V] \quad \forall \pi$	a max is $\geq$ any particular choice
$B_{\pi}[V_{\pi}]$	$= V_{\pi}$	fixed point of B_{π}
$B[V^*]$	$= V^*$	fixed point of B (Bellman optimality)

The trap. $B[V] = B_{\{\pi_{greedy}[V]\}}[V]$, but $B[V] \neq V^{\{\pi_{greedy}[V]\}}$. One sweep of *greedy backup* is not the same as committing to the greedy policy forever and computing its value. $B[V]$ looks ahead one step and then falls back on V ; $V^{\{\pi_{greedy}[V]\}}$ looks ahead forever. Confusing these two is the single most common error on this material.

PART 3 — Lecture 5: Q-values and Policy Iteration

3.1 Q-values

$Q_{\pi}(s, a)$ is the expected discounted sum of rewards when you start at s , take action a this once (even if π wouldn't have), and follow π from then on.

$$Q_{\pi}(s, a) = E \left[\sum_{t=0}^{\infty} \gamma^t R(S_t, A_t) \mid S_0 = s, A_0 = a, A_t = \pi(S_t) \text{ for } t \geq 1 \right]$$

Splitting off the first step gives the computable form:

$$Q_{\pi}(s, a) = R(s, a) + \gamma \sum_{s' \in S} P_{ss'}(a) \cdot V_{\pi}(s')$$

Why we want this. $V_{\pi}(s)$ tells you how good π is. It doesn't tell you whether π picked the right action at s . $Q_{\pi}(s, a)$ does — it lets you price each alternative action against the same yardstick V_{π} , and pick the winner.

Shape: Q_{π} is $|S| \times |A|$ — one number per (state, action) pair. Note $Q_{\pi}(s, \pi(s)) = V_{\pi}(s)$ always: the column that π actually chose reproduces V_{π} .

Computing it by hand is one line per (state, action) pair. This is the main mechanical skill of Lecture 5, so here it is spelled out on the Robot MDP with $V_{\pi} = [1, 1]$, $\gamma = 0.9$:

$$\begin{aligned}
 Q(H, \text{search}) &= R(H,s) + 0.9 [P_{HH}(s) \cdot V(H) + P_{HL}(s) \cdot V(L)] \\
 &= 1 + 0.9 [0.8 \cdot 1 + 0.2 \cdot 1] \\
 &= 1 + 0.9 (1) \\
 &= 1.9
 \end{aligned}$$

Do that once per row of the big **P** matrix. That's all “policy improvement” is.

3.2 The Policy Iteration algorithm

```

Let  $\pi_0$  be an arbitrary policy
 $k \leftarrow 0$ 
Repeat
   $\pi \leftarrow \pi_k$ 

  POLICY EVALUATION: compute  $V\pi$ 
                    (matrix inverse, or iterate  $B\pi$ )

  POLICY IMPROVEMENT: compute  $Q\pi(s,a)$  for every  $s, a$ 
                         $\pi'(s) \in \operatorname{argmax}_{a \in A} Q\pi(s,a), \quad \forall s \in S$ 

   $\pi_{k+1} \leftarrow \pi'$ 
   $k \leftarrow k + 1$ 
Until  $V^{\pi_k} = V^{\pi_{k-1}}$  (equivalently: the policy
stops changing)
Return  $\pi_k$ 

```

Two phases alternating:

$$\pi_0 \xrightarrow{\text{eval}} V\pi^0 \xrightarrow{\text{improve}} \pi_1 \xrightarrow{\text{eval}} V\pi^1 \xrightarrow{\text{improve}} \pi_2 \rightarrow \dots \rightarrow \pi^*$$

and the values are guaranteed non-decreasing all the way:

$$V\pi^0 \leq V\pi^1 \leq V\pi^2 \leq \dots \leq V\pi^*$$

Important detail on the argmax: if two actions tie, pick either — but if the *incumbent* action $\pi(s)$ is among the winners, keep it. Otherwise you can cycle forever between two equally-good policies and the termination test never fires.

3.3 The Policy Improvement Theorem

This is the theorem that makes the algorithm correct. It says: *the greedy policy is genuinely better, not just better-looking.*

Theorem. If $\pi'(s) \in \operatorname{argmax}_a Q\pi(s, a)$ for every s , then $V\pi' \geq V\pi$.

The proof has two halves. Half one is a one-line calculation; half two is Lecture 4's monotonicity applied infinitely many times.

Half 1 — $B\pi' [V\pi] \geq V\pi$

$$\begin{aligned}
 B\pi' [V\pi](s) &= R(s, \pi'(s)) + \gamma \sum_{s'} P_{ss'}(\pi'(s)) \cdot V\pi(s') \\
 &= Q\pi(s, \pi'(s)) && \leftarrow \text{by definition} \\
 \text{of } Q\pi & \\
 &= \max_a Q\pi(s, a) && \leftarrow \text{because } \pi' \\
 \text{was chosen greedily} & \\
 &\geq Q\pi(s, \pi(s)) && \leftarrow \text{a max is } \geq \\
 \text{any particular entry} & \\
 &= V\pi(s) && \leftarrow \text{because} \\
 Q\pi(s, \pi(s)) = V\pi(s) &
 \end{aligned}$$

True for every s , so $B\pi' [V\pi] \geq V\pi$. ■

What this actually says: “Deviate to π' for exactly one step, then revert to π forever. You are no worse off.” That’s a modest claim — one step only.

Half 2 — bootstrap that one step into forever

Start from $V\pi \leq B\pi' [V\pi]$ and hit both sides with $B\pi'$ repeatedly. Each application preserves the inequality by **monotonicity of $B\pi'$** (Lecture 4):

$$\begin{aligned}
 V\pi &\leq B\pi' [V\pi] \\
 &\leq B\pi' [B\pi' [V\pi]] = B^2\pi' [V\pi] && \text{(monotonicity)} \\
 &\leq B^3\pi' [V\pi] && \text{(monotonicity)} \\
 \text{again)} & \\
 &\leq \dots \\
 &\leq \lim_{k \rightarrow \infty} B^k\pi' [V\pi] \\
 &= V\pi' && \text{(convergence of } B\pi', \text{ Lecture 4)}
 \end{aligned}$$

Hence $V\pi \leq V\pi'$. ■

The idea in one sentence: deviating for one step doesn’t hurt; therefore deviating for two steps doesn’t hurt; therefore deviating forever doesn’t hurt — and deviating forever is switching to π' .

Notice how little machinery this needs. Half 1 is bookkeeping. Half 2 is $B\pi'$ being monotone plus $B\pi'$ converging. That’s why Lecture 4 comes first.

3.4 Why Policy Iteration terminates — and terminates at the optimum

Terminates. There are only finitely many deterministic policies (m^n of them). By the theorem, values never decrease. Once a policy is revisited its value would have to be both $\geq$ and $\leq$ its earlier value, so the sequence of values can never revisit a strictly-lower level — the algorithm must stop within m^n iterations. In practice it stops in a handful; you’ll see 2 and 3 in the worked examples.

Terminates at the optimum. Suppose the algorithm stops, i.e. $\pi' = \pi$. Then for every s :

$$V\pi(s) = Q\pi(s, \pi(s)) = \max_a Q\pi(s, a) = \max_a \{ R(s, a) + \gamma \sum_{s'} P_{ss'}(a) V\pi(s') \}$$

That last line is precisely the **Bellman Optimality Equation** (Lecture 6). Since V^* is the unique vector satisfying it, $V\pi = V^*$ and π is optimal. ■

So the stopping condition isn't a heuristic — it is optimality.

3.5 V^* exists (is finite)

Claim. $|V^*(s)| \leq R_{\max}/(1 - \gamma)$ for all s , where $R_{\max} = \|R\|_{\infty}$.

Proof (this is Maam's hint — use the matrix definition, and it reuses HW1 Q5 directly):

Write $M = (I - \gamma P\pi)^{-1}$. From Lecture 3 we know

$$M = \sum_{t=0}^{\infty} \gamma^t (P\pi)^t$$

Two facts about M follow immediately, both of which you proved in HW1:

1. **Every entry of M is ≥ 0** — it's a sum of non-negative terms.
2. **Every row of M sums to $1/(1-\gamma)$** — this was HW1 Q5.

Now for any policy π and any state s :

$$\begin{aligned}
 |V\pi(s)| &= \left| \sum_j M_{s_j} \cdot R\pi(j) \right| \\
 &\leq \sum_j M_{s_j} \cdot |R\pi(j)| && \text{(triangle inequality, } M \geq 0) \\
 &\leq \sum_j M_{s_j} \cdot R_{\max} && \text{(bound each reward by the max)} \\
 &= R_{\max} \cdot \sum_j M_{s_j} \\
 &= R_{\max} / (1 - \gamma) && \text{(row sums, from HW1 Q5)}
 \end{aligned}$$

This bound holds for **every** policy, and $V^*(s) = \max_{\pi} V\pi(s)$, so $|V^*(s)| \leq R_{\max}/(1-\gamma)$. ■

Sanity check on our examples ($\gamma = 0.9$, so the bound is $10 \times R_{\max}$):

MDP	$R_{\max}$	bound	actual $\max V^* $
Robot	1	10	8.475 ✓
HW1	2	20	11.743 ✓

Loose, but finite — and finiteness is the whole point. Without it, “the maximum over policies” might not be a real number.

3.6 V^* is unique

Here’s the worry. $V\pi$ is a vector, and vectors only have a **partial** order. Two policies can be genuinely incomparable — π_1 better at s_1 , π_2 better at s_2 . If that happened between two “best” policies, there’d be no single V^* and the whole theory collapses.

The theorem says it can’t happen. And the proof is constructive and rather beautiful: **you can always stitch the good halves together.**

Maam’s concrete example

Suppose both π_1 and π_2 claim to be optimal:

$$\begin{aligned}\pi_1 &= [a_1, a_1]^\top, & V\pi^1 &= [10, 5]^\top \\ \pi_2 &= [a_2, a_2]^\top, & V\pi^2 &= [5, 10]^\top\end{aligned}$$

Neither dominates. Now build a **stitched policy** — take each state's action from whichever policy is better there:

$$\begin{aligned}\pi' &= [a_1, a_2]^\top && (\pi_1 \text{'s action at } s_1, \pi_2 \text{'s action at } s_2) \\ \mathbf{V} &= [10, 10]^\top && (\text{the componentwise max: } V = \max(V\pi^1, V\pi^2))\end{aligned}$$

Step 1: show $B\pi'[V] \geq V$.

At s_1 , where π' uses π_1 's action:

$$\begin{aligned}B\pi'[V](s_1) &= R(s_1, \pi_1(s_1)) + \gamma \sum_{s'} P_{\{s_1 s'\}}(\pi_1(s_1)) \cdot V(s') \\ &\geq R(s_1, \pi_1(s_1)) + \gamma \sum_{s'} P_{\{s_1 s'\}}(\pi_1(s_1)) \cdot V\pi^1(s') \\ \leftarrow \text{since } V &\geq V\pi^1 \\ &= V\pi^1(s_1) \\ \leftarrow \text{Bellman eq. for } \pi_1 \\ &= V(s_1) \\ \leftarrow \text{by construction of } V\end{aligned}$$

At s_2 the identical argument runs with π_2 and $V\pi^2$. So $B\pi'[V] \geq V$. ✓

Step 2: bootstrap. By monotonicity of $B\pi'$, exactly as in the Policy Improvement Theorem:

$$V \leq B\pi'[V] \leq B^2\pi'[V] \leq \dots \leq \lim_k B^k\pi'[V] = V\pi'$$

So $V \leq V\pi'$.

Step 3: contradiction.

$$\begin{aligned} V\pi^1 &= [10, 5]^\top \leq V = [10, 10]^\top \leq V\pi' \\ V\pi^2 &= [5, 10]^\top \leq V = [10, 10]^\top \leq V\pi' \end{aligned}$$

So π' is at least as good as both — and strictly better than π_1 at s_2 and than π_2 at s_1 . Neither π_1 nor π_2 was maximising after all. **Contradiction.** ■

The general version

Same proof, no numbers. Given any two claimed-optimal π_1, π_2 , define

$$\begin{aligned} S_1 &= \{ s : V\pi^1(s) \geq V\pi^2(s) \} & S_2 &= S - S_1 \\ \pi'(s) &= \pi_1(s) \text{ if } s \in S_1, \quad \pi_2(s) \text{ if } s \in S_2 \\ V(s) &= V\pi^1(s) \text{ if } s \in S_1, \quad V\pi^2(s) \text{ if } s \in S_2 & (\text{i.e. } V = \text{componentwise max}) \end{aligned}$$

Steps 1–3 go through verbatim. **Conclusion: any two optimal policies have the same value vector, so V^* is unique** — and it's attained by an actual policy, not just approached.

Takeaway to carry forward: value vectors are only partially ordered in general, but the maximum over policies is always attained by a single policy. That's the non-obvious fact that makes “the optimal policy” a meaningful phrase.

PART 4 — Lecture 6: the Bellman Optimality Equation and Value Iteration

4.1 Greedy policy

The **greedy policy with respect to** V :

$$\pi_{\text{greedy}}[V](s) \in \operatorname{argmax}_a \{ R(s, a) + \gamma \sum_{s'} P_{ss'}(a) V(s') \} = \operatorname{argmax}_a \{ QV(s, a) \}$$

It's a function of a value vector. Feed it any V , get back a policy: “at each state, pick whichever action looks best assuming V is right about the future.”

Policy improvement in Lecture 5 is exactly $\pi' = \pi_{\text{greedy}}[V\pi]$ — greedy with respect to the true value of the current policy.

4.2 The Bellman Optimality Equation

$$V^*(s) = \max_{a \in A} \{ R(s, a) + \gamma \sum_{s' \in S} P_{ss'}(a) \cdot V^*(s') \}, \quad \forall s \in S$$

and an optimal policy is greedy with respect to V^* :

$$\pi^*(s) \in \operatorname{argmax}_{a \in A} \{ R(s, a) + \gamma \sum_{s' \in S} P_{ss'}(a) \cdot V^*(s') \}$$

Compare with the policy Bellman equation:

$$V\pi(s) = R(s, \pi(s)) + \gamma \sum_{s'} P_{ss'}(\pi(s)) V\pi(s') \quad \leftarrow \text{linear: } n \text{ linear equations}$$

$$V^*(s) = \max_a \{ R(s, a) + \gamma \sum_{s'} P_{ss'}(a) V^*(s') \} \quad \leftarrow \text{NON-linear: because of the max}$$

That one $\max$ changes everything. You can no longer write $V^* = (I - \gamma P)^{-1} R$ and solve it — there's no single P to use, since which row you'd pick depends on V^* , which is what you're solving for. Chicken and egg. Both Policy Iteration and Value Iteration are ways to break that circularity:

- **PI** breaks it by *guessing* the max (fix a policy) $\rightarrow$ solve the now-linear system $\rightarrow$ re-guess.
- **VI** breaks it by *iterating* rather than solving.

Reading the equation in words: the value of a state under optimal play is the value of the best single action available there, assuming you play optimally afterwards. This is **Bellman's principle of optimality**.

4.3 The operator B

$$B[V](s) = \max_{a \in A} \{ R(s, a) + \gamma \sum_{s' \in S} P_{ss'}(a) V(s') \} = \max_a QV(s, a)$$

The Bellman Optimality Equation is then just $B[V^*] = V^*$ — **V^* is the fixed point of B** .

Note B is exactly $B\pi$ with “follow π ” replaced by “take the best action”. And since a max is at least any particular element:

$$B[V] \geq B\pi[V] \quad \text{for every policy } \pi, \quad \text{with equality iff } \pi \text{ is greedy w.r.t. } V$$

4.4 Property 1: B is monotone

$$V_1 \leq V_2 \implies B[V_1] \leq B[V_2]$$

Proof. Fix any s and any action a . Since $P_{ss'}(a) \geq 0$ and $\gamma \geq 0$:

$$\begin{aligned} QV^1(s, a) &= R(s, a) + \gamma \sum_{s'} P_{ss'}(a) V_1(s') \leq R(s, a) + \gamma \sum_{s'} P_{ss'}(a) V_2(s') \\ &= QV^2(s, a) \end{aligned}$$

So $QV^1(s, \cdot) \leq QV^2(s, \cdot)$ for every action. Taking the max over a of both sides preserves $\leq$:

$$B[V_1](s) = \max_a QV^1(s, a) \leq \max_a QV^2(s, a) = B[V_2](s)$$

■

(Key step: if $f(a) \leq g(a)$ for every a , then $\max f \leq \max g$. Take a^* maximising f ; then $\max f = f(a^*) \leq g(a^*) \leq \max g$.)

4.5 Property 2: B is a contraction

$$\|B[V_1] - B[V_2]\|_\infty \leq \gamma \|V_1 - V_2\|_\infty$$

This one needs a small lemma, because B has a $\max$ in it and $\max$ is not linear — the Rn cancellation trick from Lecture 4 doesn't directly apply.

The max lemma

For any two functions f, g on a finite set:

$$\left| \max_a f(a) - \max_a g(a) \right| \leq \max_a |f(a) - g(a)|$$

Proof. Let a^* maximise f . Then

$$\begin{aligned}
 \max_a f(a) - \max_a g(a) &= f(a^*) - \max_a g(a) \\
 &\leq f(a^*) - g(a^*) && \leftarrow \text{since} \\
 \max_a g(a) \geq g(a^*) & \\
 &\leq \max_a |f(a) - g(a)|
 \end{aligned}$$

Swapping the roles of f and g bounds the other direction the same way. Taking absolute values gives the lemma. ■

(Plain-English version: two maxima can't differ by more than the largest gap between the functions anywhere.)

Proof of contraction

For any state s :

$$\begin{aligned}
 |B[V_1](s) - B[V_2](s)| &= | \max_a QV^1(s,a) - \max_a QV^2(s,a) | \\
 &\leq \max_a | QV^1(s,a) - QV^2(s,a) | \\
 \leftarrow \text{max lemma} & \\
 &= \max_a | \gamma \sum_{s'} P_{ss'}(a) [V_1(s') - V_2(s')] | \\
 &\leq \max_a \gamma \sum_{s'} P_{ss'}(a) \cdot |V_1(s') - V_2(s')| \leftarrow \text{triangle ineq.} \\
 &\leq \max_a \gamma \sum_{s'} P_{ss'}(a) \cdot \|V_1 - V_2\|_\infty \\
 &= \gamma \|V_1 - V_2\|_\infty \\
 \leftarrow \text{rows sum to 1} &
 \end{aligned}$$

True for every s , so take the max over s . ■

Consequence (Banach again): B has a **unique** fixed point, and iterating B from any starting vector converges to it. The fixed point is V^* . This is a second, independent proof that V^* is unique — Lecture 5's stitching argument

PART 4 — Lecture 6: the Bellman Optimality Equation and Value Iteration
and Lecture 6's contraction argument reach the same conclusion by different routes.

4.6 Value Iteration

```

Let  $V_0 \in \mathbb{R}^n$  be arbitrary
 $k \leftarrow 0$ 
Repeat
     $V_{k+1}(s) = B[V_k](s) = \max_a \{ R(s,a) + \gamma \sum_{s'} P_{ss'}(a) V_k(s') \}$ ,  $\forall s \in S$ 
     $k \leftarrow k + 1$ 
Until  $V_k = V_{k-1}$ 
Return  $V_k$ 

```

$$V_0 \xrightarrow{B} V_1 \xrightarrow{B} V_2 \xrightarrow{B} V_3 \rightarrow \dots \rightarrow V^*$$

Then recover the policy at the end: $\pi^* = \pi_{\text{greedy}}[V^*]$.

VI never stores a policy while it runs. It works purely with value vectors and only extracts a policy once, at the end. That's the cleanest way to remember how it differs from PI.

The intuition: v_k is the optimal k -steps-to-go value

Start from $V_0 = 0$. Then:

- $V_1(s) = \max_a R(s,a)$ — the best you can do with **one** action remaining.
- $V_2(s)$ — the best over **two** steps.
- $V_k(s)$ — the best achievable expected discounted reward in **k** steps from s .

So value iteration is literally “plan one step ahead, then two, then three...”, and the infinite-horizon answer is the limit. This also explains why V_k climbs from below when $V_0 = 0$ and all rewards are positive: you’re accounting for more and more of the future each sweep.

Cost per sweep

One sweep touches every (s, a) pair once: $O(n \cdot m \cdot n)$ in the worst case, or $O(n \cdot m \cdot d)$ where d is the average number of reachable next states. **No matrix inverse anywhere.** That’s the trade: cheap sweeps, but you need many of them.

4.7 Stopping criterion in practice

$V_k = V_{k-1}$ exactly will essentially never happen in floating point, and in theory VI converges only in the limit. So:

```

Input:  $\epsilon > 0$ 
Let  $V_0 \in \mathbb{R}^n$  be arbitrary
 $k \leftarrow 0$ 
Repeat
     $V_{k+1}(s) = B[V_k](s)$  ,  $\forall s \in S$ 
     $k \leftarrow k + 1$ 
Until  $\|V_k - V_{k-1}\|_\infty \leq \epsilon$ 
Return  $V_k$ 

```

We stop when a whole sweep moves the values by less than ϵ anywhere.

4.8 How far are we from V^* ? (Maam’s “to do” — prove this)

Claim. If we stop when $\|V_k - V_{k-1}\|_\infty \leq \epsilon$, then

$$\|V^* - V_k\|_\infty \leq \epsilon / (1 - \gamma)$$

Proof. Use two facts: $V^* = B[V^*]$ (fixed point) and $V_k = B[V_{k-1}]$ (the update rule).

$$\begin{aligned}
 \|V^* - V_k\|_\infty &= \|B[V^*] - B[V_{k-1}]\|_\infty \\
 &\leq \gamma \|V^* - V_{k-1}\|_\infty \\
 \leftarrow \text{contraction} \\
 &\leq \gamma (\|V^* - V_k\|_\infty + \|V_k - V_{k-1}\|_\infty) \\
 \leftarrow \text{triangle inequality} \\
 &\leq \gamma \|V^* - V_k\|_\infty + \gamma \varepsilon \\
 \leftarrow \text{stopping condition}
 \end{aligned}$$

Collect the $\|V^* - V_k\|_\infty$ terms on the left:

$$\begin{aligned}
 (1 - \gamma) \|V^* - V_k\|_\infty &\leq \gamma \varepsilon \\
 \boxed{\|V^* - V_k\|_\infty &\leq \gamma \varepsilon / (1 - \gamma)}
 \end{aligned}$$

■

How to use it. Turn it around: to guarantee accuracy δ , run until

$$\varepsilon \leq \delta (1 - \gamma) / \gamma$$

Worked example ($\gamma = 0.9$): to be within $\delta = 0.01$ of V^* , you need $\varepsilon \leq 0.01 \times 0.1/0.9 \approx 0.0011$. So the observed sweep-to-sweep change must be roughly **9× smaller** than the accuracy you actually want. The naive assumption “small change $\Rightarrow$ close to the answer” understates the error by a factor of $\gamma/(1-\gamma) = 9$.

This bound is tight. On the HW1 MDP with $\gamma = 0.9$, the actual error equals $\gamma\epsilon/(1-\gamma)$ to five decimals from iteration 8 onwards — see the worked example in Part 6. It's not a loose worst-case estimate; it's what actually happens.

Bonus result (not in the slides, but standard and sometimes asked): if π is greedy w.r.t. V_k , its actual value satisfies

$$\|V^* - V\pi\|_\infty \leq \frac{2\gamma}{1-\gamma} \|V^* - V_k\|_\infty$$

Loss in the policy is bounded by loss in the values, with another $\gamma/(1-\gamma)$ blow-up. In practice VI usually finds the optimal policy far earlier than the optimal values — you'll see this vividly in Part 6.

PART 5 — Policy Iteration vs Value Iteration

	Policy Iteration	Value Iteration
Iterates over	policies $\pi_0, \pi_1, \dots$	value vectors $V_0, V_1, \dots$
Per iteration	evaluate (solve $n \times n$ system) + improve	one sweep of B
Cost per iteration	$O(n^3 + n^2m)$ — expensive	$O(n^2m)$ — cheap
Iterations needed	very few (2–3 typical; $\leq m^n$ guaranteed)	many; γ -dependent, infinite in theory
Terminates exactly?	Yes , in finite steps	No — only ϵ -close
Intermediate V	is a real policy's value V^{π_k}	generally not any policy's value
Monotone?	$V^{\pi^0} \leq V^{\pi^1} \leq \dots$ guaranteed	not in general (depends on V_0)
Stopping test	policy stops changing	$\ V_k - V_{k-1}\ _\infty \leq \epsilon$
Output	π^* directly	V^* ; extract $\pi^* = \text{argreedy}[V^*]$

How to choose. Small $n \rightarrow$ Policy Iteration, because the exact solve is affordable and you converge in a handful of iterations. Huge $n \rightarrow$ Value Iteration, because you can't invert anything. In between $\rightarrow$ **Modified Policy Iteration**: run PI, but do the evaluation step approximately with a few sweeps of B_{π} instead of an exact solve. VI is the extreme case of exactly one sweep.

Both are grounded in the same two properties — monotonicity and contraction — applied to B^π and B respectively. That's the unifying takeaway of Lectures 4–6.

Preview: Linear Programming (the third algorithm, not yet covered)

For completeness, since Maam listed three, V^* can also be found by solving

```

minimise     $\sum_s V(s)$ 
subject to   $V(s) \geq R(s,a) + \gamma \sum_{s'} P_{ss'}(a) V(s')$       for
every  $s \in S, a \in A$ 

```

n variables, $n \cdot m$ constraints. Any feasible V is an upper bound on V^* ; minimising pushes it down until it hits V^* exactly. Expect this in a later lecture — don't worry about it now.

PART 6 — Fully worked examples

6.1 Robot MDP — Policy Iteration

Recall ($\gamma = 0.9$, states H, L):

$$\begin{array}{c}
 \mathbf{P} = \begin{array}{c} \begin{array}{cc} & \begin{array}{c} s'=H \\ s'=L \end{array} \\ \begin{array}{l} (H, \text{search}) \\ (H, \text{wait}) \\ (L, \text{search}) \\ (L, \text{wait}) \\ (L, \text{recharge}) \end{array} & \begin{bmatrix} 0.8 & 0.2 \\ 1.0 & 0.0 \\ 0.3 & 0.7 \\ 0.0 & 1.0 \\ 1.0 & 0.0 \end{bmatrix} \end{array}
 \end{array}
 \end{array}
 \quad
 \mathbf{R} = \begin{array}{c} \begin{array}{c} R \\ \begin{bmatrix} 1 \\ 0.1 \\ -0.2 \\ 0.1 \\ 0 \end{bmatrix} \end{array}
 \end{array}$$

Iteration 0 — $\pi_0 = (\text{wait}, \text{wait})$

Evaluate: $V\pi^0 = [1, 1]^T$ (computed in the Lectures 1–4 guide).

Improve: compute $Q\pi^0(s, a) = R(s, a) + 0.9 [P_{sH}(a) \cdot 1 + P_{sL}(a) \cdot 1]$ for all five rows.

$$\begin{aligned}
 Q(H, \text{search}) &= 1 + 0.9[0.8(1) + 0.2(1)] = 1 + 0.9(1.0) = 1.9 \quad \leftarrow \max \\
 Q(H, \text{wait}) &= 0.1 + 0.9[1.0(1) + 0.0(1)] = 0.1 + 0.9(1.0) = 1.0 \\
 Q(L, \text{search}) &= -0.2 + 0.9[0.3(1) + 0.7(1)] = -0.2 + 0.9(1.0) = 0.7 \\
 Q(L, \text{wait}) &= 0.1 + 0.9[0.0(1) + 1.0(1)] = 0.1 + 0.9(1.0) = 1.0 \quad \leftarrow \max \\
 Q(L, \text{recharge}) &= 0 + 0.9[1.0(1) + 0.0(1)] = 0 + 0.9(1.0) = 0.9
 \end{aligned}$$

$\pi_1 = (\text{search}, \text{wait})$. Changed at H.

Iteration 1 — $\pi_1 = (\text{search}, \text{wait})$

Evaluate: $V\pi^1 = [4.2143, 1]^\top$ (this is Maam's π_4 ; slide says $[4.214, 1]^\top$).

Improve:

$$\begin{aligned}
 Q(H, \text{search}) &= 1 + 0.9[0.8(4.2143) + 0.2(1)] = 1 + 0.9(3.5714) = 4.2143 \leftarrow \max \\
 Q(H, \text{wait}) &= 0.1 + 0.9[4.2143] = 0.1 + 3.7929 = 3.8929 \\
 \\
 Q(L, \text{search}) &= -0.2 + 0.9[0.3(4.2143) + 0.7(1)] = -0.2 + 0.9(1.9643) = 1.5679 \\
 Q(L, \text{wait}) &= 0.1 + 0.9[1] = 0.1 + 0.9 = 1.0000 \\
 Q(L, \text{recharge}) &= 0 + 0.9[4.2143] = 3.7929 \\
 &\leftarrow \max
 \end{aligned}$$

$\pi_2 = (\text{search}, \text{recharge})$. Changed at L .

Note $Q(H, \text{search}) = 4.2143 = V\pi^1(H)$ exactly — the incumbent action always reproduces its own value. Free arithmetic check on every iteration.

Iteration 2 — $\pi_2 = (\text{search}, \text{recharge})$

Evaluate: $V\pi^2 = [8.4746, 7.6271]^\top$.

Improve:

$$\begin{aligned}
Q(H, \text{search}) &= 1 + 0.9[0.8(8.4746) + 0.2(7.6271)] = 1 + 0.9(8.3051) = 8.4746 \leftarrow \max \\
Q(H, \text{wait}) &= 0.1 + 0.9[8.4746] = 7.7271
\end{aligned}$$

$$\begin{aligned}
Q(L, \text{search}) &= -0.2 + 0.9[0.3(8.4746) + 0.7(7.6271)] = -0.2 + 0.9(7.8814) = 6.8932 \\
Q(L, \text{wait}) &= 0.1 + 0.9[7.6271] = 6.9644 \\
Q(L, \text{recharge}) &= 0 + 0.9[8.4746] = 7.6271 \leftarrow \max
\end{aligned}$$

Policy unchanged $\rightarrow$ stop.

$$\pi^* = (\text{search}, \text{recharge}), \quad V^* = [8.4746, 7.6271]^T$$

✓ Matches Maam's Lecture 2 slide, where π_2 was circled as optimal.

Bellman optimality check: $Q^*(H, \text{search}) = 8.4746 = V^*(H)$ and $Q^*(L, \text{recharge}) = 7.6271 = V^*(L)$. Both states satisfy $V^*(s) = \max_a Q^*(s, a)$. ✓

Summary trace:

k	π_k	V^{π_k}
0	(wait, wait)	[1.0000, 1.0000]
1	(search, wait)	[4.2143, 1.0000]
2	(search, recharge)	[8.4746, 7.6271]
3	stable	—

Three evaluations. Values increased at every step, as the theorem promised.

6.2 Robot MDP — Value Iteration

$$\begin{aligned} B[V](H) &= \max\{ 1 + 0.9(0.8 V_H + 0.2 V_L) , \quad 0.1 + 0.9 V_H \} \\ B[V](L) &= \max\{ -0.2 + 0.9(0.3 V_H + 0.7 V_L) , \quad 0.1 + 0.9 V_L , \quad 0 + 0.9 V_H \} \end{aligned}$$

$$V_0 = [0, 0]^T$$

$$\begin{aligned} V_1(H) &= \max\{ 1 + 0, \quad 0.1 + 0 \} &= \max\{ 1, 0.1 \} \\ &= 1 \quad [\text{search}] \\ V_1(L) &= \max\{ -0.2 + 0, \quad 0.1 + 0, \quad 0 \} &= \max\{ -0.2, \\ &0.1, 0 \} &= 0.1 \quad [\text{wait}] \end{aligned}$$

$$V_1 = [1, 0.1]^T$$

$$\begin{aligned} V_2(H) &= \max\{ 1 + 0.9(0.8 \cdot 1 + 0.2 \cdot 0.1), \quad 0.1 + 0.9 \cdot 1 \} \\ &= \max\{ 1 + 0.9(0.82), \quad 1.0 \} = \max\{ 1.738, 1.0 \} \\ &= 1.738 \quad [\text{search}] \\ V_2(L) &= \max\{ -0.2 + 0.9(0.3 \cdot 1 + 0.7 \cdot 0.1), \quad 0.1 + 0.9(0.1), \\ &0.9(1) \} \\ &= \max\{ -0.2 + 0.9(0.37), \quad 0.19, \quad 0.9 \} = \max\{ 0.133, \\ &0.19, 0.9 \} = 0.9 \quad [\text{recharge}] \end{aligned}$$

$$V_2 = [1.738, 0.9]^T$$

$$\begin{aligned} V_3(H) &= \max\{ 1 + 0.9(0.8 \cdot 1.738 + 0.2 \cdot 0.9), \quad 0.1 + 0.9(1.738) \} \\ &= \max\{ 1 + 0.9(1.5704), \quad 1.6642 \} = \max\{ 2.4134, 1.6642 \} \\ &= 2.4134 \\ V_3(L) &= \max\{ -0.2 + 0.9(0.3 \cdot 1.738 + 0.7 \cdot 0.9), \quad 0.1 + 0.9(0.9), \\ &0.9(1.738) \} \\ &= \max\{ 0.8363, \quad 0.91, \quad 1.5642 \} \\ &= 1.5642 \end{aligned}$$

$$V_3 = [2.4134, 1.5642]^T$$

Continuing:

k	$V_k(H)$	$V_k(L)$	greedy policy	$\ V_k - V_{k-1}\ _\infty$
0	0	0	—	—
1	1.0000	0.1000	(search, wait)	1.0000
2	1.7380	0.9000	(search, recharge)	0.8000
3	2.4134	1.5642	(search, recharge)	0.6754
5	3.5648	2.7173	(search, recharge)	0.5456
10	5.5754	4.7279	(search, recharge)	0.3221
43	8.3850	7.5375	(search, recharge)	0.0100
∞	8.4746	7.6271	(search, recharge)	0

Two things to notice, and they're the point of this example:

1. **The optimal policy appears at $k = 2$** and never changes — but the values are still wrong by 6.7 at that point, and it takes **43 sweeps** to get ϵ down to 0.01. Policy convergence is much faster than value convergence.
2. **Compare with Policy Iteration: 3 iterations vs 43 sweeps.** That's the trade-off in the table in Part 5, made concrete. PI's iterations cost more each, but there are dramatically fewer of them.

Error bound check at $k = 43$: $\epsilon = 0.00995$, so the bound gives $\|V^* - V_{43}\|_\infty \leq 0.9(0.00995)/0.1 = 0.0896$. Actual error: $8.4746 - 8.3850 = 0.0896$. **Exactly the bound** — see the note in §4.8 about tightness.

6.3 HW1 MDP — all 8 policies, and a surprise

Your homework MDP ($\gamma = 0.9$):

$(s_1, a_1): 0.9 \rightarrow s_1, 0.1 \rightarrow s_2, \mathbf{R} = 1$ $(s_1, a_2): 1.0 \rightarrow s_1, \mathbf{R} =$
 1
 $(s_2, a_1): 0.9 \rightarrow s_2, 0.1 \rightarrow s_3, \mathbf{R} = 1$ $(s_2, a_2): 1.0 \rightarrow s_1, \mathbf{R} =$
 2
 $(s_3, a_1): 0.9 \rightarrow s_3, 0.1 \rightarrow s_1, \mathbf{R} = 1$ $(s_3, a_2): 1.0 \rightarrow s_1, \mathbf{R} =$
 -1

All eight deterministic policies, evaluated:

π	$V\pi(s_1)$	$V\pi(s_2)$	$V\pi(s_3)$
(a_1, a_1, a_1)	10.0000	10.0000	10.0000
(a_1, a_1, a_2)	9.4377	8.8129	7.4939
(a_1, a_2, a_1)	10.8257	11.7431	10.3911
(a_1, a_2, a_2)	10.8257	11.7431	8.7431
(a_2, a_1, a_1)	10.0000	10.0000	10.0000
(a_2, a_1, a_2)	10.0000	9.0526	8.0000
(a_2, a_2, a_1)	10.0000	11.0000	10.0000
(a_2, a_2, a_2)	10.0000	11.0000	8.0000

The surprise: the policy from your homework, (a_2, a_2, a_2) with $V_\pi = [10, 11, 8]$, is **not** optimal. The optimal policy is $\pi^* = (a_1, a_2, a_1)$ with

$$V^* = [10.8257, 11.7431, 10.3911]^T$$

It beats (a_2, a_2, a_2) in **every** state. Notice also that π^* mixes actions — no single action is right everywhere. And notice (a_1, a_2, a_1) dominates all seven others componentwise, exactly as the uniqueness theorem in §3.6 guarantees it must.

Why does a_2 win at s_2 but lose at s_1 and s_3 ? At s_2 , a_2 pays a $+2$ bonus for jumping to s_1 — worth the move. At s_3 , a_2 costs -1 to make the same jump, and s_3 isn't bad enough to justify paying that. At s_1 , a_1 and a_2 pay the same reward, but a_1 gives a 0.1 chance of drifting to s_2 — the most valuable state — so a_1 edges it.

6.4 HW1 MDP — Policy Iteration

Start from the homework policy $\pi_0 = (a_2, a_2, a_2)$, so you can reuse work you've already done.

Iteration 0 — $\pi_0 = (a_2, a_2, a_2)$, $V\pi^0 = [10, 11, 8]^T$

$$\begin{aligned} Q(s_1, a_1) &= 1 + 0.9[0.9(10) + 0.1(11) + 0(8)] = 1 + 0.9(10.1) \\ &= 10.09 \quad \leftarrow \max \\ Q(s_1, a_2) &= 1 + 0.9[1.0(10)] = 1 + 9 \\ &= 10.00 \end{aligned}$$

$$\begin{aligned} Q(s_2, a_1) &= 1 + 0.9[0(10) + 0.9(11) + 0.1(8)] = 1 + 0.9(10.7) \\ &= 10.63 \\ Q(s_2, a_2) &= 2 + 0.9[1.0(10)] = 2 + 9 \\ &= 11.00 \quad \leftarrow \max \end{aligned}$$

$$\begin{aligned} Q(s_3, a_1) &= 1 + 0.9[0.1(10) + 0(11) + 0.9(8)] = 1 + 0.9(8.2) \\ &= 8.38 \quad \leftarrow \max \\ Q(s_3, a_2) &= -1 + 0.9[1.0(10)] = -1 + 9 \\ &= 8.00 \end{aligned}$$

$\pi_1 = (a_1, a_2, a_1)$. Changed at s_1 and s_3 .

Iteration 1 — $\pi_1 = (a_1, a_2, a_1)$

Evaluate. Select rows (s_1, a_1) , (s_2, a_2) , (s_3, a_1) :

$$P\pi = \begin{matrix} & s_1 & s_2 & s_3 \\ \begin{matrix} s_1 \\ s_2 \\ s_3 \end{matrix} & \begin{bmatrix} 0.9 & 0.1 & 0 \\ 1.0 & 0 & 0 \\ 0.1 & 0 & 0.9 \end{bmatrix} \end{matrix} \quad R\pi = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix}$$

Recursive equations:

$$\begin{aligned} V_1 &= 1 + 0.9(0.9 V_1 + 0.1 V_2) = 1 + 0.81 V_1 + 0.09 V_2 \\ V_2 &= 2 + 0.9 V_1 \\ V_3 &= 1 + 0.9(0.1 V_1 + 0.9 V_3) = 1 + 0.09 V_1 + 0.81 V_3 \end{aligned}$$

Substitute V_2 into the first:

$$\begin{aligned}
 V_1 &= 1 + 0.81 V_1 + 0.09(2 + 0.9 V_1) \\
 &= 1 + 0.81 V_1 + 0.18 + 0.081 V_1 \\
 &= 1.18 + 0.891 V_1
 \end{aligned}$$

$$0.109 V_1 = 1.18 \quad \Rightarrow \quad V_1 = 1.18/0.109 = 10.8257$$

Then

$$\begin{aligned}
 V_2 &= 2 + 0.9(10.8257) = 11.7431 \\
 0.19 V_3 &= 1 + 0.09(10.8257) = 1.9743 \quad \Rightarrow \quad V_3 = 10.3911
 \end{aligned}$$

Improve.

$$\begin{aligned}
 Q(s_1, a_1) &= 1 + 0.9[0.9(10.8257) + 0.1(11.7431)] = 1 + \\
 0.9(10.9174) &= 10.8257 \quad \leftarrow \max \\
 Q(s_1, a_2) &= 1 + 0.9(10.8257) = 10.7431 \\
 \\
 Q(s_2, a_1) &= 1 + 0.9[0.9(11.7431) + 0.1(10.3911)] = 1 + \\
 0.9(11.6079) &= 11.4471 \\
 Q(s_2, a_2) &= 2 + 0.9(10.8257) = 11.7431 \\
 &\leftarrow \max \\
 \\
 Q(s_3, a_1) &= 1 + 0.9[0.1(10.8257) + 0.9(10.3911)] = 1 + \\
 0.9(10.4346) &= 10.3911 \quad \leftarrow \max \\
 Q(s_3, a_2) &= -1 + 0.9(10.8257) = 8.7431
 \end{aligned}$$

Policy unchanged $\rightarrow$ stop. $\pi^* = (a_1, a_2, a_1)$, $V^* = [10.8257, 11.7431, 10.3911]^T$.

Two iterations. And note every $Q(s, \pi^*(s))$ equals $V^*(s)$ exactly — the Bellman optimality equation, satisfied. ✓

6.5 HW1 MDP — Value Iteration, and the tightness of the error bound

$$\begin{aligned} B[V](s_1) &= \max\{ 1 + 0.9(0.9V_1 + 0.1V_2) , \quad 1 + 0.9V_1 \} \\ B[V](s_2) &= \max\{ 1 + 0.9(0.9V_2 + 0.1V_3) , \quad 2 + 0.9V_1 \} \\ B[V](s_3) &= \max\{ 1 + 0.9(0.1V_1 + 0.9V_3) , \quad -1 + 0.9V_1 \} \end{aligned}$$

$$V_0 = [0, 0, 0]^T$$

$$\begin{aligned} V_1(s_1) &= \max\{1, 1\} = 1 && \text{(tie — both actions pay 1)} \\ V_1(s_2) &= \max\{1, 2\} = 2 && [a_2] \\ V_1(s_3) &= \max\{1, -1\} = 1 && [a_1] \end{aligned}$$

$V_1 = [1, 2, 1]^T$ — and this is just $\max_a R(s, a)$, the best one-step reward. ✓

$$\begin{aligned} V_2(s_1) &= \max\{ 1 + 0.9(0.9 \cdot 1 + 0.1 \cdot 2) , \quad 1 + 0.9 \cdot 1 \} = \max\{ 1.99, 1.9 \} = 1.99 \quad [a_1] \\ V_2(s_2) &= \max\{ 1 + 0.9(0.9 \cdot 2 + 0.1 \cdot 1) , \quad 2 + 0.9 \cdot 1 \} = \max\{ 2.71, 2.9 \} = 2.90 \quad [a_2] \\ V_2(s_3) &= \max\{ 1 + 0.9(0.1 \cdot 1 + 0.9 \cdot 1) , \quad -1 + 0.9 \cdot 1 \} = \max\{ 1.90, -0.1 \} = 1.90 \quad [a_1] \end{aligned}$$

$$V_2 = [1.99, 2.90, 1.90]^T$$

Full trace:

k	$V_k(s_1)$	$V_k(s_2)$	$V_k(s_3)$	greedy	$\epsilon =$ $\ V_k - V_{k-1}\ _\infty$	bound $\gamma\epsilon/(1-\gamma)$	actual
1	1.0000	2.0000	1.0000	(a ₁ , a ₂ , a ₁)	2.00000	18.0000	9.8257
2	1.9900	2.9000	1.9000	(a ₁ , a ₂ , a ₁)	0.99000	8.9100	8.8431

PART 6 — Fully worked examples

3	2.8729	3.7910	2.7181	(a_1, a_2, a_1)	0.89100	8.0190	7.9528
5	4.3840	5.3014	4.1329	(a_1, a_2, a_1)	0.71581	6.4423	6.4417
8	6.1297	7.0471	5.7926	(a_1, a_2, a_1)	0.52178	4.6960	4.6960
10	—	—	—	(a_1, a_2, a_1)	0.42264	3.8038	3.8038
15	—	—	—	(a_1, a_2, a_1)	0.24957	2.2461	2.2461
25	—	—	—	(a_1, a_2, a_1)	0.08702	0.7832	0.7832
∞	10.8257	11.7431	10.3911	(a_1, a_2, a_1)	0	0	0

Three lessons, all visible in this one table:

1. **The greedy policy is optimal from sweep 1** — before the values are anywhere near right. VI finds the policy almost immediately and then spends dozens of sweeps polishing *numbers*.
2. **The error bound becomes an equality** from about $k = 5$ onward. $\gamma\epsilon/(1-\gamma)$ is not a pessimistic estimate here — it's exactly the error. So take it seriously when choosing ϵ .
3. ϵ **badly understates the true error**. At $k = 25$ a sweep moves values by only 0.087 , but you're still 0.78 away from V^* — nine times worse than the sweep size suggests. That factor of nine is $\gamma/(1-\gamma)$.

PART 7 — Practice problems

Cover the answers first.

P1. Compute Q_π on the Robot MDP

Given $\pi = (\text{search}, \text{search})$ with $V_\pi = [6.073, 3.891]^\top$ and $\gamma = 0.9$, compute all five $Q_\pi(s, a)$ values and find the improved policy.



Answer

P2. Prove the max lemma

Show $|\max_a f(a) - \max_a g(a)| \leq \max_a |f(a) - g(a)|$.



Answer

P3. Why does policy improvement give $V_{\pi'} \geq V_\pi$, not just $B_{\pi'}[V_\pi] \geq V_\pi$?



Answer

P4. Value iteration by hand

Run two sweeps of value iteration from $V_0 = [0, 0]^\top$ on this MDP ($\gamma = 0.5$):

$(s_1, a): 1.0 \rightarrow s_2, \quad R = 4$
 $(s_2, a): 1.0 \rightarrow s_1, \quad R = 0$

$(s_1, b): 1.0 \rightarrow s_1, \quad R = 3$
 $(s_2, b): 1.0 \rightarrow s_2, \quad R = 1$



Answer

P5. How many sweeps?

With $\gamma = 0.95$, you want $\|V^* - V_k\|_\infty \leq 0.01$. What stopping threshold ϵ do you need?

▶

Answer

P6. Conceptual short answers

- ▶ Q: What's the difference between $B_\pi[V]$ and V_π ?
- ▶ Q: Why is the Bellman Optimality Equation nonlinear, and why does that matter?
- ▶ Q: Is $B[V] = V^{\pi_{\text{greedy}}[V]}$?
- ▶ Q: Why does PI terminate in finitely many steps but VI doesn't?
- ▶ Q: In PI, why must policy evaluation be exact?
- ▶ Q: Can two different policies both be optimal?
- ▶ Q: Where exactly is $\gamma < 1$ used?

PART 8 — Cheat sheet

FIVE BACKUPS (all: immediate reward + $\gamma \times$ average over next states)

$V\pi(s) = R(s, \pi(s)) + \gamma \sum P_{ss'}(\pi(s)) V\pi(s')$ true value of π

$Q\pi(s, a) = R(s, a) + \gamma \sum P_{ss'}(a) V\pi(s')$ take **a** once, then π

$B\pi[V](s) = R(s, \pi(s)) + \gamma \sum P_{ss'}(\pi(s)) V(s')$ one sweep of "follow π "

$QV(s, a) = R(s, a) + \gamma \sum P_{ss'}(a) V(s')$ take **a** once, then trust V

$B[V](s) = \max_a \{ R(s, a) + \gamma \sum P_{ss'}(a) V(s') \}$ one sweep of "act greedily"

RELATIONS

$Q\pi(s, \pi(s)) = V\pi(s)$ $B\pi[V](s) = QV(s, \pi(s))$ $B[V]$

$(s) = \max_a QV(s, a)$

$B\pi[V\pi] = V\pi$ $B[V^*] = V^*$ $B[V]$

$\geq B\pi[V] \quad \forall \pi$

$\pi_{\text{greedy}}[V](s) \in \operatorname{argmax}_a QV(s, a)$ $B[V] =$

$B_{\{\pi_{\text{greedy}}[V]\}}[V] \neq V^{\{\pi_{\text{greedy}}[V]\}}$

PROPERTIES (hold for BOTH $B\pi$ and B)

monotone: $V_1 \leq V_2 \Rightarrow B[V_1] \leq B[V_2]$

contraction: $\|B[V_1] - B[V_2]\|_\infty \leq \gamma \|V_1 - V_2\|_\infty$

$\Rightarrow$ unique fixed point, convergence from any V_0 , $\|fix - V_k\|_\infty \leq \gamma^k \|fix - V_0\|_\infty$

POLICY ITERATION

π_0 arbitrary

repeat:

eval: $V\pi = (I - \gamma P\pi)^{-1} R\pi$

improve: $\pi'(s) \in \operatorname{argmax}_a Q\pi(s, a)$

$V_{\{k-1\}}\|_\infty \leq \epsilon$

until $\pi' = \pi$

guarantees $V\pi^0 \leq V\pi^1 \leq \dots \leq V^*$

$\gamma\epsilon/(1-\gamma)$

finite termination ($\leq m^n$ iterations)

exact

VALUE ITERATION

V_0 arbitrary

repeat:

$V_{\{k+1\}} = B[V_k]$

until $\|V_k -$

$\pi^* = \pi_{\text{greedy}}[V^*]$

$\|V^* - V_k\|_\infty \leq$

geometric, never

BELLMAN OPTIMALITY EQUATION (nonlinear – the max is why)

$V^*(s) = \max_a \{ R(s, a) + \gamma \sum_{s'} P_{ss'}(a) V^*(s') \}$

$\pi^*(s) \in \operatorname{argmax}_a \{ R(s, a) + \gamma \sum_{s'} P_{ss'}(a) V^*(s') \}$

KEY FACTS

V^* exists: $|V^*(s)| \leq R_{\max}/(1-\gamma)$ [rows of $(I-\gamma P\pi)^{-1}$
 are ≥ 0 and sum to $1/(1-\gamma)$]
 V^* unique: stitch π_1 and π_2 on $S_1, S_2 \rightarrow B\pi' [V] \geq V \rightarrow V \leq V\pi'$
 $\rightarrow$ contradiction
 $\|Px\|_\infty \leq \|x\|_\infty$ for stochastic P [the lemma behind
 every contraction proof]
 $|\max f - \max g| \leq \max |f - g|$ [the lemma that makes
 B a contraction]
 to get accuracy δ , stop at $\varepsilon \leq \delta(1-\gamma)/\gamma$

WORKED ANSWERS

Robot MDP: $\pi^* = (\text{search}, \text{recharge})$, $V^* = [8.4746, 7.6271]$
 PI: 3 iterations. VI: 43 sweeps to $\varepsilon = 0.01$.
 HW1 MDP: $\pi^* = (a_1, a_2, a_1)$, $V^* = [10.8257, 11.7431, 10.3911]$
 PI: 2 iterations. (a_2, a_2, a_2) from the homework
 is NOT optimal.

Common traps

Trap	Correct
$B[V] = V^{\pi_{\text{greedy}}[V]}$	$B[V] = B_{\{\pi_{\text{greedy}}[V]\}}[V]$ — one backup, not the full value
Treating the Bellman optimality equation as linear	The $\max$ makes it nonlinear; no matrix inverse
$Q_{\pi}(s, a) = R(s, a) + \gamma \sum_{s'} P_{ss'}(a) Q_{\pi}(s', a)$	Back up onto $V_{\pi}(s')$, not Q_{π} . (The Q -form needs $\max_{a'}$ or $\pi(s')$ inside.)
Using V_{π} from the <i>previous</i> policy when improving	Improve using the value of the <i>current</i> policy — evaluate first, then improve
Concluding $V_k \approx V^*$ because ϵ is small	Error is up to $\gamma\epsilon/(1-\gamma)$ — a factor 9 bigger at $\gamma = 0.9$, 19 at $\gamma = 0.95$
Assuming a policy that's best in one state is best overall	Optimality is componentwise; but §3.6 guarantees a single policy dominates everywhere
Flipping the argmax on a tie every iteration	Keep the incumbent action when it ties, or PI may never terminate
Forgetting to extract π^* after VI	VI returns V^* ; you still need one greedy sweep to get π^*
Assuming the “obvious” policy is optimal	The HW1 MDP is the counterexample — (a_2, a_2, a_2) loses to (a_1, a_2, a_1) everywhere

Reading

Sutton & Barto, Chapter 4 (Dynamic Programming) maps onto these lectures almost exactly:

- **4.1** Policy Evaluation → Lecture 4
- **4.2** Policy Improvement → Lecture 5 (the Policy Improvement Theorem is Section 4.2)
- **4.3** Policy Iteration → Lecture 5
- **4.4** Value Iteration → Lecture 6
- **4.6** Generalized Policy Iteration → the unifying view of both

Chapter 3.5–3.8 covers Q^π , V^* , Q^* and the Bellman Optimality Equation. Free at incompleteideas.net/book/the-book.html.